

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Double-click (or enter) to edit

```
import os

PROJECT_DIR = '/content/drive/MyDrive/sports_classification'

os.makedirs(PROJECT_DIR, exist_ok=True)

print("Project folder created:")
print(PROJECT_DIR)
```

Project folder created:
/content/drive/MyDrive/sports_classification

```
import os

PROJECT_DIR = '/content/drive/MyDrive/sports_classification'

os.makedirs(PROJECT_DIR, exist_ok=True)

print("Project folder created:")
print(PROJECT_DIR)
```

Project folder created:
/content/drive/MyDrive/sports_classification

```
import os

print(os.listdir('/content'))
```

['.config', '90.jpg', '62.jpg', '98.jpg', '35.jpg', '70.jpg', '86.jpg', '85.jpg']

```
import zipfile
import os

zip_path = '/content/archive.zip'
extract_path = '/content/data'

os.makedirs(extract_path, exist_ok=True)

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
```

```
zip_ref.extractall(extract_path)
```

```
print("Dataset extracted successfully!")
```

Dataset extracted successfully!

```
!find /content/data -maxdepth 2 -type d | head -50
```

```
/content/data
/content/data/train
/content/data/train/judo
/content/data/train/bike polo
/content/data/train/axe throwing
/content/data/train/luge
/content/data/train/swimming
/content/data/train/frisbee
/content/data/train/tug of war
/content/data/train/trapeze
/content/data/train/archery
/content/data/train/javelin
/content/data/train/bmx
/content/data/train/hurdles
/content/data/train/jousting
/content/data/train/golf
/content/data/train/speed skating
/content/data/train/horseshoe pitching
/content/data/train/horse racing
/content/data/train/nascar racing
/content/data/train/rock climbing
/content/data/train/air hockey
/content/data/train/figure skating women
/content/data/train/formula 1 racing
/content/data/train/log rolling
/content/data/train/mushing
/content/data/train/water cycling
/content/data/train/baton twirling
/content/data/train/figure skating pairs
/content/data/train/barell racing
/content/data/train/track bicycle
/content/data/train/steer wrestling
/content/data/train/rowing
/content/data/train/ampute football
/content/data/train/pole climbing
/content/data/train/fly fishing
/content/data/train/arm wrestling
/content/data/train/lacrosse
/content/data/train/rugby
/content/data/train/disc golf
/content/data/train/ultimate
/content/data/train/rings
/content/data/train/hang gliding
/content/data/train/pole vault
/content/data/train/sailboat racing
/content/data/train/giant slalom
/content/data/train/surfing
```

```

/content/data/train/fencing
/content/data/train/tennis
/content/data/train/ice yachting

```

```
!find /content/data -name "sports.csv"
```

```
/content/data/sports.csv
```

```

import pandas as pd

df = pd.read_csv('/content/sports.csv')

print(df.head())
print()
print("Number of rows:", len(df))
print("Number of classes:", df['labels'].nunique())

```

	class id	filepaths	labels	data set
0	0	train/air hockey/001.jpg	air hockey	train
1	0	train/air hockey/002.jpg	air hockey	train
2	0	train/air hockey/003.jpg	air hockey	train
3	0	train/air hockey/004.jpg	air hockey	train
4	0	train/air hockey/005.jpg	air hockey	train

Number of rows: 14493

Number of classes: 100

```

print("Number of classes:", df['labels'].nunique())

print("\nSports classes:")
for i, sport in enumerate(sorted(df['labels'].unique()), 1):
    print(i, sport)

```

Number of classes: 100

Sports classes:

- 1 air hockey
- 2 ampute football
- 3 archery
- 4 arm wrestling
- 5 axe throwing
- 6 balance beam
- 7 barell racing
- 8 baseball
- 9 basketball
- 10 baton twirling
- 11 bike polo
- 12 billiards
- 13 bmx
- 14 bobsled
- 15 bowling
- 16 boxing
- 17 bull riding

```
18 bungee jumping
19 canoe slamon
20 cheerleading
21 chuckwagon racing
22 cricket
23 croquet
24 curling
25 disc golf
26 fencing
27 field hockey
28 figure skating men
29 figure skating pairs
30 figure skating women
31 fly fishing
32 football
33 formula 1 racing
34 frisbee
35 gaga
36 giant slalom
37 golf
38 hammer throw
39 hang gliding
40 harness racing
41 high jump
42 hockey
43 horse jumping
44 horse racing
45 horseshoe pitching
46 hurdles
47 hydroplane racing
48 ice climbing
49 ice yachting
50 jai alai
51 javelin
52 jousting
53 judo
54 lacrosse
55 log rolling
```

```
class_counts = df['labels'].value_counts()

print(class_counts)
```

```
labels
football          201
formula 1 racing  200
nascar racing     199
baseball          184
hockey            182
...
horseshoe pitching 112
arm wrestling      109
wingsuit flying    108
ultimate           107
```

```
sky surfing          69
Name: count, Length: 100, dtype: int64
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 25))

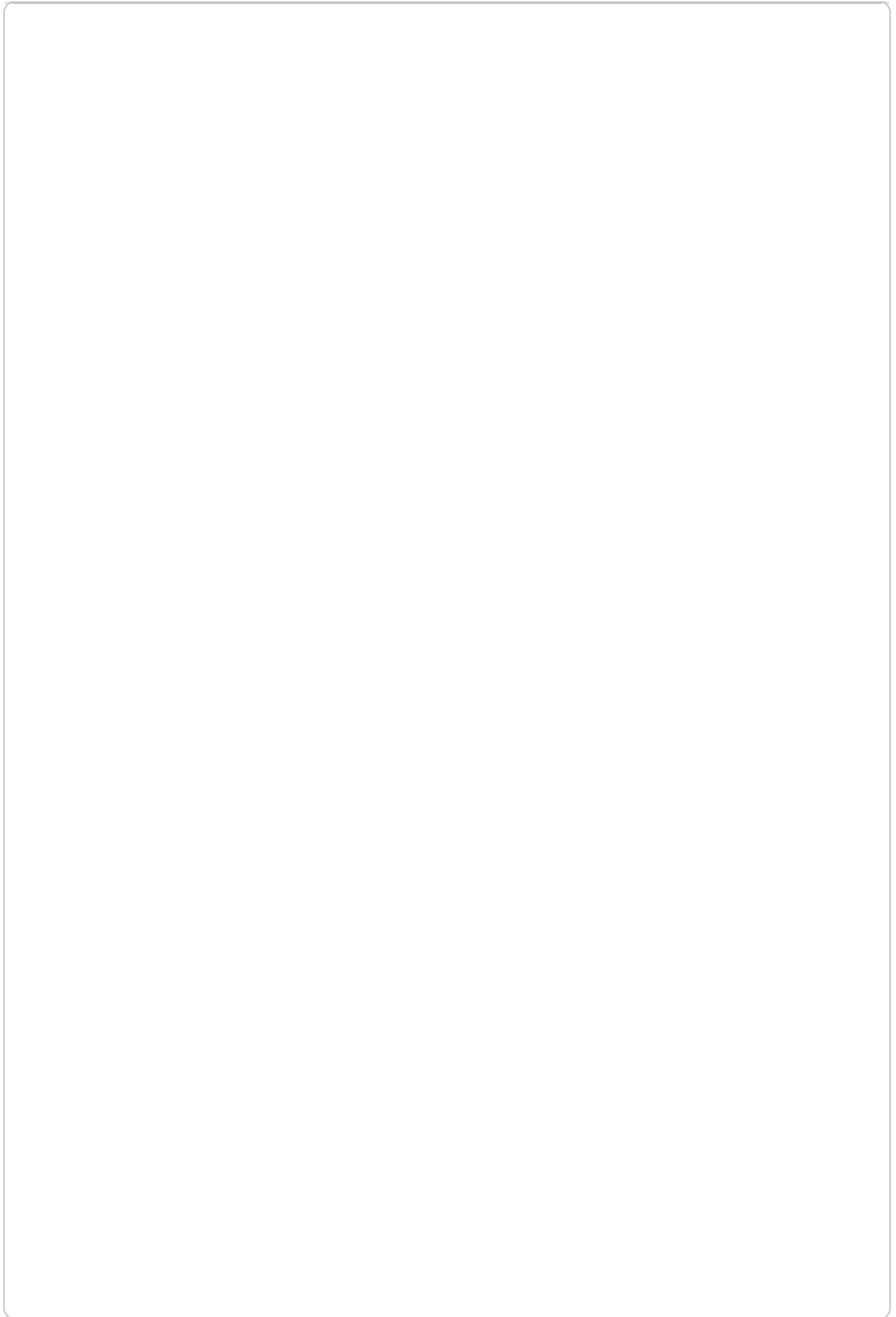
class_counts.sort_values().plot(kind='barh')

plt.xlabel('Number of Images')
plt.ylabel('Sports Class')
plt.title('Class Distribution of 100 Sports')

plt.tight_layout()

plt.savefig(
    f'{PROJECT_DIR}/class_distribution.png',
    dpi=300,
    bbox_inches='tight'
)

plt.show()
```



Class Distribution of 100 Sports

```
print("Total images:", len(df))
print("Unique image paths:", df['filepaths'].nunique())
print("Number of classes:", df['labels'].nunique())
```

```
Total images: 14493
Unique image paths: 14493
Number of classes: 100
```

```
from PIL import Image
import os
```

```
sample_df = df.sample(
    min(200, len(df)),
    random_state=42
)
```

```
dimensions = []
```

```
for filepath in sample_df['filepaths']:
    full_path = os.path.join('/content/data', filepath)

    try:
        with Image.open(full_path) as img:
            dimensions.append(img.size)
    except Exception as e:
        print("Could not read:", full_path)
```

```
from collections import Counter
```

```
print(Counter(dimensions).most_common(20))
```

```
[((224, 224), 200)]
```

```
# Install ImageHash
%pip install -q ImageHash
```

```
import imagehash
from PIL import Image
import os
```

```
hashes = {}
duplicates = []
invalid_files = []
```

```
# Check every file
for filepath in df['filepaths']:
```

```
    full_path = os.path.join('/content/data', filepath)
```

```

try:
    # Try opening the file as an image
    with Image.open(full_path) as img:
        img.verify()

    # Re-open after verify()
    with Image.open(full_path) as img:
        h = str(imagehash.phash(img))

    # Check duplicate
    if h in hashes:
        duplicates.append(
            (hashes[h], full_path)
        )
    else:
        hashes[h] = full_path

except Exception as e:
    invalid_files.append({
        "filepath": filepath,
        "error": str(e)
    })

print("=" * 60)
print("DATASET QUALITY CHECK")
print("=" * 60)

print("Total records:", len(df))
print("Valid images:", len(hashes))
print("Invalid/non-image files:", len(invalid_files))
print("Duplicate/near-duplicate pairs:", len(duplicates))

print("\nInvalid files:")
for item in invalid_files:
    print("-", item["filepath"])

print("\nUnique valid images:", len(hashes))

```

```

=====
DATASET QUALITY CHECK
=====
Total records: 14493
Valid images: 14492
Invalid/non-image files: 1
Duplicate/near-duplicate pairs: 0

Invalid files:
- train/high jump/159.lnk

Unique valid images: 14492

```

```
# Remove invalid/non-image files

invalid_paths = [item["filepath"] for item in invalid_files]

df_clean = df[~df['filepath'].isin(invalid_paths)].copy()

print("=" * 50)
print("CLEANED DATASET")
print("=" * 50)

print("Original records:", len(df))
print("Clean records:", len(df_clean))
print("Removed records:", len(df) - len(df_clean))
print("Number of classes:", df_clean['labels'].nunique())
```

```
=====
CLEANED DATASET
=====
Original records: 14493
Clean records: 14492
Removed records: 1
Number of classes: 100
```

```
# Class distribution

class_counts = df_clean['labels'].value_counts().sort_index()

print("Number of classes:", len(class_counts))
print("\nImages in each class:")
print(class_counts)
```

```
Number of classes: 100

Images in each class:
labels
air hockey          122
ampute football     122
archery             142
arm wrestling        109
axe throwing         123
...
water polo          140
weightlifting        144
wheelchair basketball 116
wheelchair racing    129
wingsuit flying      108
Name: count, Length: 100, dtype: int64
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 25))
```

```
class_counts.sort_values().plot(kind='barh')

plt.xlabel("Number of Images")
plt.ylabel("Sports Class")
plt.title("Class Distribution - 100 Sports Image Classification Dataset")

plt.tight_layout()

plt.show()
```


Class Distribution - 100 Sports Image Classification Dataset

```

clean_csv_path = '/content/drive/MyDrive/sports_classification/cleaned_sports.c

df_clean.to_csv(clean_csv_path, index=False)

print("Cleaned CSV saved successfully!")
print(clean_csv_path)

```

```

Cleaned CSV saved successfully!
/content/drive/MyDrive/sports_classification/cleaned_sports.csv

```

```

from sklearn.model_selection import train_test_split

# 70% Training, 30% Temporary
train_df, temp_df = train_test_split(
    df_clean,
    test_size=0.30,
    stratify=df_clean['labels'],
    random_state=42
)

# Split the remaining 30% into:
# 15% Validation and 15% Testing
val_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    stratify=temp_df['labels'],
    random_state=42
)

print("=" * 60)
print("70 / 15 / 15 STRATIFIED DATASET SPLIT")
print("=" * 60)

print("Total images:", len(df_clean))
print("Training images:", len(train_df))
print("Validation images:", len(val_df))
print("Testing images:", len(test_df))

print("\nPercentages:")

total = len(df_clean)

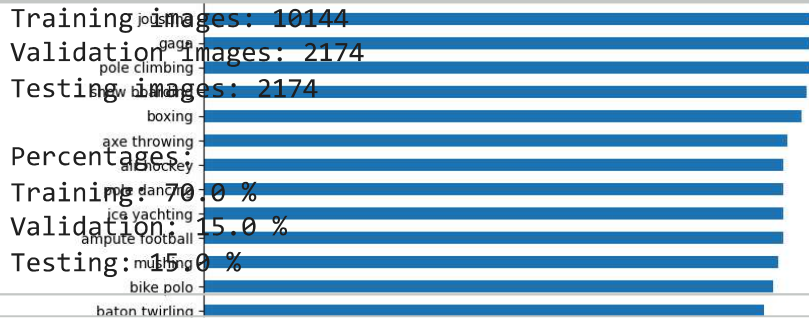
print("Training:", round(len(train_df) / total * 100, 2), "%")
print("Validation:", round(len(val_df) / total * 100, 2), "%")
print("Testing:", round(len(test_df) / total * 100, 2), "%")

```

```

=====
70 / 15 / 15 STRATIFIED DATASET SPLIT
=====
Total images: 14492

```



Training images: 10144
 Validation images: 2174
 Testing images: 2174
 Percentages:
 Training: 70.0 %
 Validation: 15.0 %
 Testing: 15.0 %

```
print("Train:", len(train_df))
print("Validation:", len(val_df))
print("Test:", len(test_df))

print("\nClasses:")
print("Train classes:", train_df['labels'].nunique())
print("Validation classes:", val_df['labels'].nunique())
print("Test classes:", test_df['labels'].nunique())
```

```
Train: 10144
Validation: 2174
Test: 2174

Classes:
Train classes: 100
Validation classes: 100
Test classes: 100
```

```
import torch
from torchvision import transforms

# Image size
IMG_SIZE = 224

# Training augmentation
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(
        IMG_SIZE,
        scale=(0.8, 1.0)
    ),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(
        brightness=0.2,
        contrast=0.2,
        saturation=0.2,
        hue=0.05
    ),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

```
])

# Validation preprocessing
val_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

# Test preprocessing
test_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

print("Transforms created successfully!")
```

Transforms created successfully!

```
import os
import shutil

# Create the split folders
base_split_path = "/content/split"

os.makedirs(base_split_path, exist_ok=True)

def create_image_split(dataframe, split_name):
    print(f"\nCreating {split_name} split...")

    count = 0

    for _, row in dataframe.iterrows():

        source_path = os.path.join(
            "/content/data",
            row["filepaths"]
        )

        class_name = str(row["labels"])

        destination_dir = os.path.join(
            base_split_path,
```

```
        split_name,
        class_name
    )

    os.makedirs(destination_dir, exist_ok=True)

    # Keep original filename
    destination_path = os.path.join(
        destination_dir,
        os.path.basename(row["filepaths"])
    )

    shutil.copy2(source_path, destination_path)

    count += 1

    if count % 1000 == 0:
        print(f"{count} images copied...")

    print(f"{split_name} complete: {count} images")

# Create all three splits
create_image_split(train_df, "train")
create_image_split(val_df, "val")
create_image_split(test_df, "test")

print("\nAll dataset splits created successfully!")
```

```
Creating train split...
1000 images copied...
2000 images copied...
3000 images copied...
4000 images copied...
5000 images copied...
6000 images copied...
7000 images copied...
8000 images copied...
9000 images copied...
10000 images copied...
train complete: 10144 images
```

```
Creating val split...
1000 images copied...
2000 images copied...
val complete: 2174 images
```

```
Creating test split...
1000 images copied...
2000 images copied...
test complete: 2174 images
```

All dataset splits created successfully!

```
import os

print("Train folder exists:", os.path.exists("/content/split/train"))
print("Validation folder exists:", os.path.exists("/content/split/val"))
print("Test folder exists:", os.path.exists("/content/split/test"))

print("\nNumber of train classes:",
      len(os.listdir("/content/split/train")))

print("Number of validation classes:",
      len(os.listdir("/content/split/val")))

print("Number of test classes:",
      len(os.listdir("/content/split/test")))
```

```
Train folder exists: True
Validation folder exists: True
Test folder exists: True

Number of train classes: 100
Number of validation classes: 100
Number of test classes: 100
```

```
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader

# Dataset directories
train_path = "/content/split/train"
val_path = "/content/split/val"
test_path = "/content/split/test"

# Create datasets
train_dataset = ImageFolder(
    train_path,
    transform=train_transform
)

val_dataset = ImageFolder(
    val_path,
    transform=val_transform
)

test_dataset = ImageFolder(
    test_path,
    transform=test_transform
)

print("=" * 50)
```

```
print("DATASET INFORMATION")
print("=" * 50)

print("Train images:", len(train_dataset))
print("Validation images:", len(val_dataset))
print("Test images:", len(test_dataset))

print("Number of classes:", len(train_dataset.classes))
```

```
=====
DATASET INFORMATION
=====
Train images: 9889
Validation images: 2164
Test images: 2165
Number of classes: 100
```

```
from pathlib import Path

def count_images(directory):
    extensions = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

    return sum(
        1 for file in Path(directory).rglob("*")
        if file.suffix.lower() in extensions
    )

train_count = count_images(train_path)
valid_count = count_images(val_path)
test_count = count_images(test_path)

total_count = train_count + valid_count + test_count

print("Training images :", train_count)
print("Validation images:", valid_count)
print("Testing images  :", test_count)
print("TOTAL IMAGES    :", total_count)
```

```
Training images : 9889
Validation images: 2164
Testing images  : 2165
TOTAL IMAGES    : 14218
```

```
classes = sorted([
    d.name for d in Path(train_path).iterdir()
    if d.is_dir()
])

NUM_CLASSES = len(classes)

print("Number of classes:", NUM_CLASSES)
```

```
for i, cls in enumerate(classes):  
    print(i, cls)
```

```
42 horse jumping  
43 horse racing  
44 horseshoe pitching  
45 hurdles  
46 hydroplane racing  
47 ice climbing  
48 ice yachting  
49 jai alai  
50 javelin
```

```
94 water_cycling
95 water polo
96 weightlifting
97 wheelchair basketball
98 wheelchair racing
99 wingsuit_flying
```

```
def get_class_counts(directory):
    counts = {}

    for cls in classes:
        class_dir = Path(directory) / cls

        if class_dir.exists():
            counts[cls] = count_images(class_dir)
        else:
            counts[cls] = 0

    return counts

train_counts = get_class_counts(train_path)

distribution_df = pd.DataFrame({
    "Class": list(train_counts.keys()),
    "Images": list(train_counts.values())
})

distribution_df = distribution_df.sort_values(
    "Images",
    ascending=False
)

distribution_df.head(20)
```

	Class	Images	
32	formula 1 racing	137	
31	football	137	
58	nascar racing	137	
7	baseball	127	
41	hockey	124	
8	basketball	123	
71	rugby	121	
59	olympic wrestling	121	
18	canoe slamon	120	
68	roller derby	120	
69	rollerblade racing	116	
40	high jump	115	
36	golf	115	
26	field hockey	114	
29	figure skating women	114	
85	swimming	114	
64	polo	114	
92	uneven bars	114	
66	rings	113	
28	figure skating pairs	111	

Next steps: [Generate code with distribution_df](#)

[New interactive sheet](#)

```
os.makedirs("results", exist_ok=True)

distribution_df.to_csv(
    "results/class_distribution.csv",
    index=False
)
```

```
plt.figure(figsize=(24, 8))

plt.bar(
    distribution_df["Class"],
```

[illegible]

https://colab.research.google.com/drive/1RKvzWqIFaT4y9aVjh_PNI dwASX6v4FWo#scrollTo=ZURrYkqgWSO0&printMode=true

)

dimension_df.head(20)

	Dimensions	Count	
0	(224, 224)	14218	

```
print("Unique image dimensions:")
print(len(dimension_counter))

print("\nMost common dimensions:")
print(dimension_df.head())
```

Unique image dimensions:
1

Most common dimensions:

	Dimensions	Count
0	(224, 224)	14218

```
import random

fig, axes = plt.subplots(
    4, 5,
    figsize=(15, 12)
)

selected_classes = random.sample(classes, 20)

for ax, cls in zip(axes.flatten(), selected_classes):

    class_dir = Path(train_path) / cls

    image_files = list(class_dir.glob("*.jpg"))

    if len(image_files) == 0:
        image_files = list(class_dir.glob("*"))

    image_path = random.choice(image_files)

    image = Image.open(image_path).convert("RGB")

    ax.imshow(image)
    ax.set_title(cls)
    ax.axis("off")

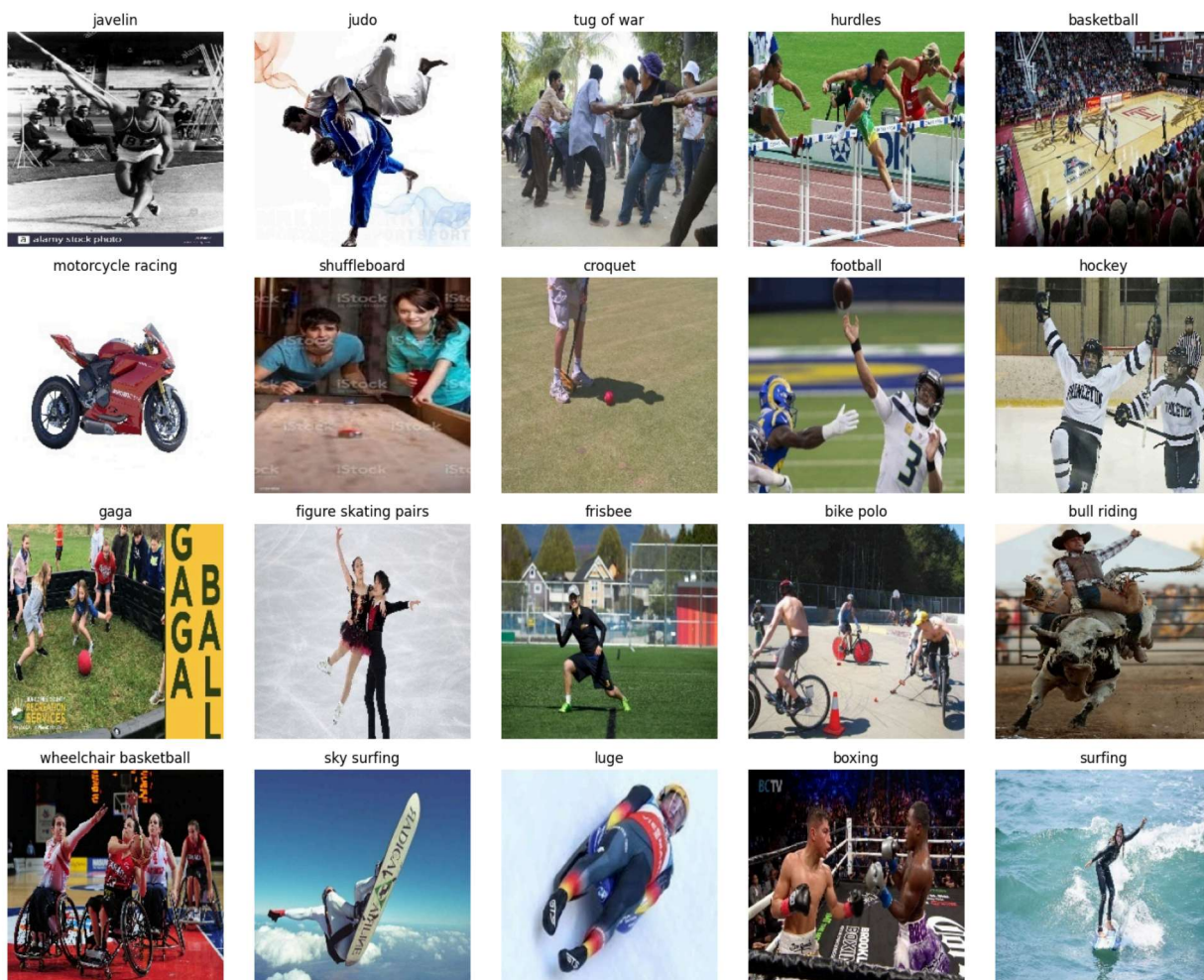
plt.tight_layout()

plt.savefig(
    "results/sample_images.png"
```

```

    dpi=300
)
plt.show()

```



```

IMG_SIZE = 224

train_transform = transforms.Compose([

    transforms.Resize((IMG_SIZE, IMG_SIZE)),

    transforms.RandomHorizontalFlip(
        p=0.5
    ),

    transforms.RandomRotation(
        15
    ),

    transforms.RandomResizedCrop(
        IMG_SIZE,
        scale=(0.8, 1.0)
    ),

```

```
        transforms.ColorJitter(
            brightness=0.2,
            contrast=0.2,
            saturation=0.2
        ),

        transforms.ToTensor(),

        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )
    ])
```

```
IMG_SIZE = 224

train_transform = transforms.Compose([

    transforms.Resize((IMG_SIZE, IMG_SIZE)),

    transforms.RandomHorizontalFlip(
        p=0.5
    ),

    transforms.RandomRotation(
        15
    ),

    transforms.RandomResizedCrop(
        IMG_SIZE,
        scale=(0.8, 1.0)
    ),

    transforms.ColorJitter(
        brightness=0.2,
        contrast=0.2,
        saturation=0.2
    ),

    transforms.ToTensor(),

    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

```
import torchvision.datasets as datasets

train_dataset = datasets.ImageFolder(
```

```
        train_path,
        transform=train_transform
    )

    valid_dataset = datasets.ImageFolder(
        val_path,
        transform=test_transform
    )

    test_dataset = datasets.ImageFolder(
        test_path,
        transform=test_transform
    )

    print("Training:", len(train_dataset))
    print("Validation:", len(valid_dataset))
    print("Testing:", len(test_dataset))

    print("Classes:", len(train_dataset.classes))
```

```
Training: 9889
Validation: 2164
Testing: 2165
Classes: 100
```

```
BATCH_SIZE = 32

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=2,
    pin_memory=True
)

valid_loader = DataLoader(
    valid_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2,
    pin_memory=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2,
    pin_memory=True
)
```

```
images, labels = next(iter(train_loader))
```

```
print("Batch image shape:", images.shape)
print("Batch label shape:", labels.shape)
```

```
Batch image shape: torch.Size([32, 3, 224, 224])
```

```
Batch label shape: torch.Size([32])
```

```
import torchvision.models as models
```

```
import torch.nn as nn
```

```
def create_model(model_name, num_classes):
```

```
    if model_name == "ResNet18":
```

```
        model = models.resnet18(
            weights=models.ResNet18_Weights.DEFAULT
        )
```

```
        model.fc = nn.Linear(
            model.fc.in_features,
            num_classes
        )
```

```
    elif model_name == "ResNet50":
```

```
        model = models.resnet50(
            weights=models.ResNet50_Weights.DEFAULT
        )
```

```
        model.fc = nn.Linear(
            model.fc.in_features,
            num_classes
        )
```

```
    elif model_name == "DenseNet121":
```

```
        model = models.densenet121(
            weights=models.DenseNet121_Weights.DEFAULT
        )
```

```
        model.classifier = nn.Linear(
            model.classifier.in_features,
            num_classes
        )
```

```
    elif model_name == "EfficientNetB0":
```

```
        model = models.efficientnet_b0(
            weights=models.EfficientNet_B0_Weights.DEFAULT
        )
```

```
        model.classifier[1] = nn.Linear(
            model.classifier[1].in_features,
            num_classes
        )

    elif model_name == "MobileNetV3":

        model = models.mobilenet_v3_large(
            weights=models.MobileNet_V3_Large_Weights.DEFAULT
        )

        model.classifier[3] = nn.Linear(
            model.classifier[3].in_features,
            num_classes
        )

    elif model_name == "ConvNeXtTiny":

        model = models.convnext_tiny(
            weights=models.ConvNeXt_Tiny_Weights.DEFAULT
        )

        model.classifier[2] = nn.Linear(
            model.classifier[2].in_features,
            num_classes
        )

    elif model_name == "ViT":

        model = models.vit_b_16(
            weights=models.ViT_B_16_Weights.DEFAULT
        )

        model.heads.head = nn.Linear(
            model.heads.head.in_features,
            num_classes
        )

    else:
        raise ValueError(
            "Unknown model"
        )

    return model
```

```
import torchvision.models as models
import torch.nn as nn

def create_model(model_name, num_classes):
```

```
if model_name == "ResNet18":

    model = models.resnet18(
        weights=models.ResNet18_Weights.DEFAULT
    )

    model.fc = nn.Linear(
        model.fc.in_features,
        num_classes
    )

elif model_name == "ResNet50":

    model = models.resnet50(
        weights=models.ResNet50_Weights.DEFAULT
    )

    model.fc = nn.Linear(
        model.fc.in_features,
        num_classes
    )

elif model_name == "DenseNet121":

    model = models.densenet121(
        weights=models.DenseNet121_Weights.DEFAULT
    )

    model.classifier = nn.Linear(
        model.classifier.in_features,
        num_classes
    )

elif model_name == "EfficientNetB0":

    model = models.efficientnet_b0(
        weights=models.EfficientNet_B0_Weights.DEFAULT
    )

    model.classifier[1] = nn.Linear(
        model.classifier[1].in_features,
        num_classes
    )

elif model_name == "MobileNetV3":

    model = models.mobilenet_v3_large(
        weights=models.MobileNet_V3_Large_Weights.DEFAULT
    )
```

```
        model.classifier[3] = nn.Linear(
            model.classifier[3].in_features,
            num_classes
        )

    elif model_name == "ConvNeXtTiny":

        model = models.convnext_tiny(
            weights=models.ConvNeXt_Tiny_Weights.DEFAULT
        )

        model.classifier[2] = nn.Linear(
            model.classifier[2].in_features,
            num_classes
        )

    elif model_name == "ViT":

        model = models.vit_b_16(
            weights=models.ViT_B_16_Weights.DEFAULT
        )

        model.heads.head = nn.Linear(
            model.heads.head.in_features,
            num_classes
        )

    else:
        raise ValueError(
            "Unknown model"
        )

    return model
```

```
EPOCHS = 10
LEARNING_RATE = 1e-4
WEIGHT_DECAY = 1e-4
```

```
import torch.optim as optim

def get_optimizer(model):

    return optim.AdamW(
        model.parameters(),
        lr=LEARNING_RATE,
        weight_decay=WEIGHT_DECAY
    )
```

```
import torch.nn as nn

criterion = nn.CrossEntropyLoss()
```

```
def get_scheduler(optimizer):

    return optim.lr_scheduler.CosineAnnealingLR(
        optimizer,
        T_max=EPOCHS
    )
```

```
def train_one_epoch(
    model,
    loader,
    optimizer,
    criterion
):

    model.train()

    running_loss = 0
    correct = 0
    total = 0

    progress = tqdm(
        loader,
        desc="Training",
        leave=False
    )

    for images, labels in progress:

        images = images.to(DEVICE)
        labels = labels.to(DEVICE)

        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(
            outputs,
            labels
        )

        loss.backward()

        optimizer.step()

        running_loss += (
            loss.item() *
```

```
        images.size(0)
    )

    _, predicted = torch.max(
        outputs,
        1
    )

    total += labels.size(0)

    correct += (
        predicted == labels
    ).sum().item()

epoch_loss = running_loss / total
epoch_accuracy = correct / total

return epoch_loss, epoch_accuracy
```

```
def train_one_epoch(
    model,
    loader,
    optimizer,
    criterion
):

    model.train()

    running_loss = 0
    correct = 0
    total = 0

    progress = tqdm(
        loader,
        desc="Training",
        leave=False
    )

    for images, labels in progress:

        images = images.to(DEVICE)
        labels = labels.to(DEVICE)

        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(
            outputs,
            labels
```

```
)

loss.backward()

optimizer.step()

running_loss += (
    loss.item() *
    images.size(0)
)

_, predicted = torch.max(
    outputs,
    1
)

total += labels.size(0)

correct += (
    predicted == labels
).sum().item()

epoch_loss = running_loss / total
epoch_accuracy = correct / total

return epoch_loss, epoch_accuracy
```

```
def validate(
    model,
    loader,
    criterion
):

    model.eval()

    running_loss = 0
    correct = 0
    total = 0

    with torch.no_grad():

        for images, labels in tqdm(
            loader,
            desc="Validation",
            leave=False
        ):

            images = images.to(DEVICE)
            labels = labels.to(DEVICE)
```

```
        outputs = model(images)

        loss = criterion(
            outputs,
            labels
        )

        running_loss += (
            loss.item() *
            images.size(0)
        )

        _, predicted = torch.max(
            outputs,
            1
        )

        total += labels.size(0)

        correct += (
            predicted == labels
        ).sum().item()

    loss = running_loss / total
    accuracy = correct / total

    return loss, accuracy
```

```
def train_model(model_name):

    print("\n" + "="*60)
    print("TRAINING:", model_name)
    print("="*60)

    model = create_model(
        model_name,
        NUM_CLASSES
    )

    model = model.to(DEVICE)

    optimizer = get_optimizer(model)

    scheduler = get_scheduler(
        optimizer
    )

    best_val_accuracy = 0

    history = {
```

```
"train_loss": [],
"train_accuracy": [],
"val_loss": [],
"val_accuracy": []
}

os.makedirs(
    "results/checkpoints",
    exist_ok=True
)

for epoch in range(EPOCHS):

    print(
        f"\nEpoch {epoch+1}/{EPOCHS}"
    )

    train_loss, train_acc = train_one_epoch(
        model,
        train_loader,
        optimizer,
        criterion
    )

    val_loss, val_acc = validate(
        model,
        valid_loader,
        criterion
    )

    scheduler.step()

    history["train_loss"].append(
        train_loss
    )

    history["train_accuracy"].append(
        train_acc
    )

    history["val_loss"].append(
        val_loss
    )

    history["val_accuracy"].append(
        val_acc
    )

    print(
        f"Train Loss: {train_loss:.4f}"
    )
```

```

        print(
            f"Train Accuracy: {train_acc:.4f}"
        )

        print(
            f"Val Loss: {val_loss:.4f}"
        )

        print(
            f"Val Accuracy: {val_acc:.4f}"
        )

        if val_acc > best_val_accuracy:

            best_val_accuracy = val_acc

            torch.save(
                model.state_dict(),
                f"results/checkpoints/{model_name}_best.pth"
            )

            print("Best model saved.")

    return model, history

```

```

import json
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

# -----
# Training configuration
# -----
EPOCHS = 10
LEARNING_RATE = 1e-4
WEIGHT_DECAY = 1e-4

# Make sure results folder exists
import os
os.makedirs("results", exist_ok=True)
os.makedirs("results/checkpoints", exist_ok=True)

# -----
# Optimizer
# -----
def get_optimizer(model):

    return optim.AdamW(

```

```
        model.parameters(),
        lr=LEARNING_RATE,
        weight_decay=WEIGHT_DECAY
    )

# -----
# Scheduler
# -----
def get_scheduler(optimizer):

    return optim.lr_scheduler.CosineAnnealingLR(
        optimizer,
        T_max=EPOCHS
    )

# -----
# Loss
# -----
criterion = nn.CrossEntropyLoss()

# -----
# Train one epoch
# -----
def train_one_epoch(
    model,
    loader,
    optimizer,
    criterion
):

    model.train()

    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader:

        images = images.to(DEVICE)
        labels = labels.to(DEVICE)

        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(
            outputs,
            labels
        )
```

```
        loss.backward()

        optimizer.step()

        running_loss += (
            loss.item() * images.size(0)
        )

        predictions = outputs.argmax(
            dim=1
        )

        correct += (
            predictions == labels
        ).sum().item()

        total += labels.size(0)

    epoch_loss = running_loss / total
    epoch_accuracy = correct / total

    return epoch_loss, epoch_accuracy

# -----
# Validation
# -----
def validate(
    model,
    loader,
    criterion
):

    model.eval()

    running_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():

        for images, labels in loader:

            images = images.to(DEVICE)
            labels = labels.to(DEVICE)

            outputs = model(images)

            loss = criterion(
                outputs,
```

```

        labels
    )

    running_loss += (
        loss.item() * images.size(0)
    )

    predictions = outputs.argmax(
        dim=1
    )

    correct += (
        predictions == labels
    ).sum().item()

    total += labels.size(0)

epoch_loss = running_loss / total
epoch_accuracy = correct / total

return epoch_loss, epoch_accuracy

# -----
# Train model
# -----
def train_model(model_name):

    print("=" * 60)
    print(f"TRAINING: {model_name}")
    print("=" * 60)

    model = create_model(
        model_name,
        NUM_CLASSES
    )

    model = model.to(DEVICE)

    optimizer = get_optimizer(model)

    scheduler = get_scheduler(
        optimizer
    )

    best_val_accuracy = 0.0

    history = {
        "train_loss": [],
        "train_accuracy": [],
        "val_loss": [],
        "val accuracy": []
    }

```

```
}

for epoch in range(EPOCHS):

    print(
        f"\nEpoch {epoch + 1}/{EPOCHS}"
    )

    train_loss, train_acc = train_one_epoch(
        model,
        train_loader,
        optimizer,
        criterion
    )

    val_loss, val_acc = validate(
        model,
        valid_loader,
        criterion
    )

    scheduler.step()

    history["train_loss"].append(
        train_loss
    )

    history["train_accuracy"].append(
        train_acc
    )

    history["val_loss"].append(
        val_loss
    )

    history["val_accuracy"].append(
        val_acc
    )

    print(
        f"Train Loss      : {train_loss:.4f}"
    )

    print(
        f"Train Accuracy : {train_acc * 100:.2f}%"
    )

    print(
        f"Val Loss        : {val_loss:.4f}"
    )

    print(
```

```

        print(
            f"Val Accuracy    : {val_acc * 100:.2f}%"
        )

    # Save best model
    if val_acc > best_val_accuracy:

        best_val_accuracy = val_acc

        torch.save(
            model.state_dict(),
            f"results/checkpoints/"
            f"{model_name}_best.pth"
        )

        print(
            "✓ Best model saved"
        )

    return model, history

# -----
# Train selected models
# -----
all_histories = {}

for model_name in MODEL_NAMES:

    model, history = train_model(
        model_name
    )

    all_histories[
        model_name
    ] = history

    # Save history
    with open(
        f"results/{model_name}_history.json",
        "w"
    ) as f:

        json.dump(
            history,
            f,
            indent=4
        )

    del model

    if torch.cuda.is_available():

```

```
torch.cuda.empty_cache()

# -----
# Plot accuracy
# -----
for model_name in MODEL_NAMES:

    history = all_histories[
        model_name
    ]

    plt.figure(
        figsize=(10, 6)
    )

    plt.plot(
        history["train_accuracy"],
        label="Training Accuracy"
    )

    plt.plot(
        history["val_accuracy"],
        label="Validation Accuracy"
    )

    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")

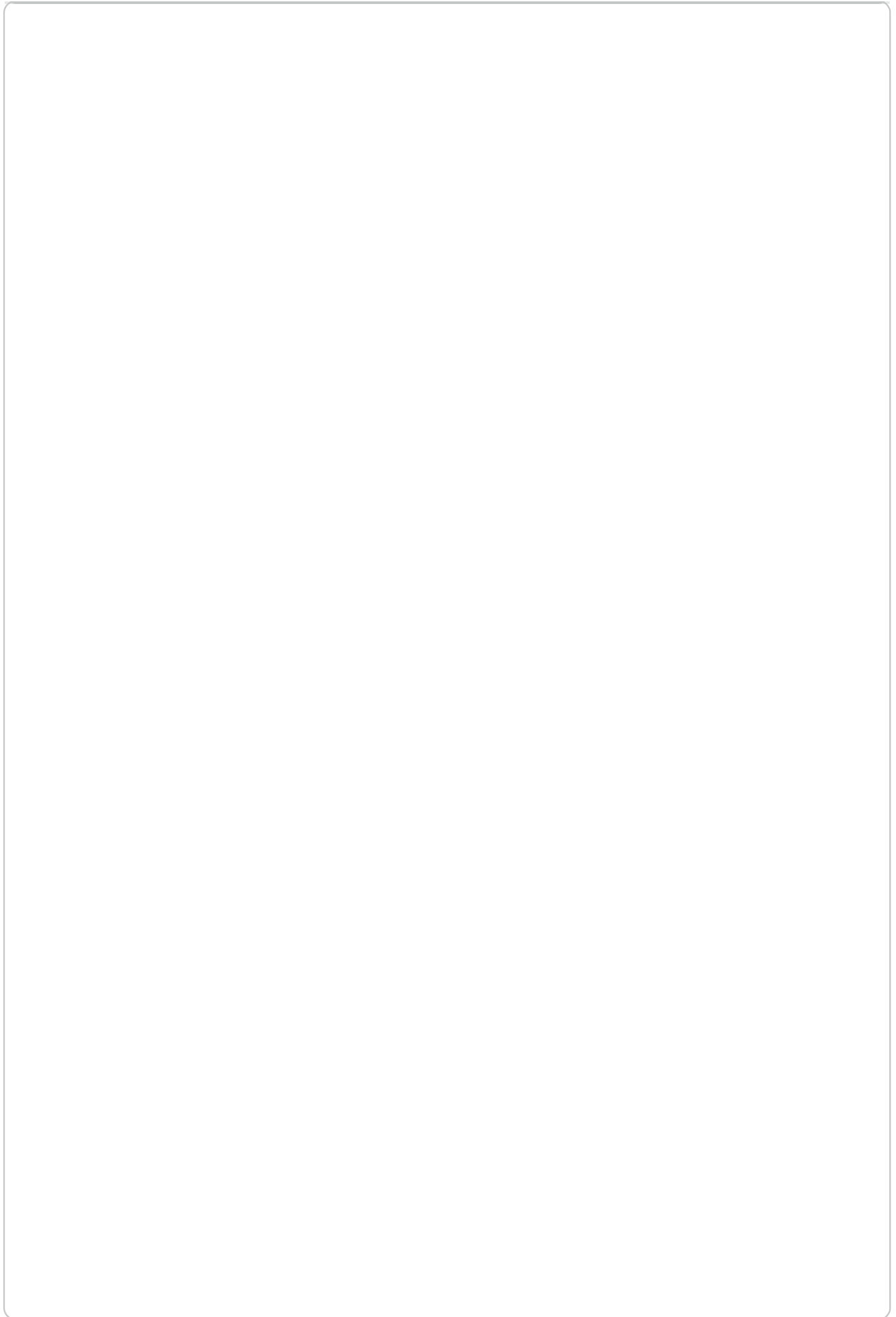
    plt.title(
        f"{model_name} - Accuracy"
    )

    plt.legend()
    plt.grid()

    plt.tight_layout()

    plt.savefig(
        f"results/"
        f"{model_name}_accuracy_curve.png",
        dpi=300
    )

    plt.show()
```



```
=====
TRAINING: ResNet18
=====
```

```
Epoch 1/10
Train Loss      : 2.3904
Train Accuracy  : 52.90%
Val Loss        : 0.9662
Val Accuracy    : 80.50%
✓ Best model saved
```

```
Epoch 2/10
Train Loss      : 0.8598
Train Accuracy  : 84.67%
Val Loss        : 0.5990
Val Accuracy    : 86.78%
✓ Best model saved
```

```
Epoch 3/10
Train Loss      : 0.4742
Train Accuracy  : 91.85%
Val Loss        : 0.4846
Val Accuracy    : 88.77%
✓ Best model saved
```

```
Epoch 4/10
Train Loss      : 0.2899
Train Accuracy  : 95.32%
Val Loss        : 0.3978
Val Accuracy    : 90.48%
✓ Best model saved
```

```
Epoch 5/10
Train Loss      : 0.1909
Train Accuracy  : 97.49%
Val Loss        : 0.3637
```

```
MODEL_NAMES = ['ResNet18']
```

```
MODEL_NAMES = [
    'ResNet18',
    'ResNet50',
    'DenseNet121',
    'EfficientNetB0',
    'MobileNetV3',
    'ConvNeXtTiny',
    'ViT'
]
```

```
Val Accuracy    : 92.56%
```

```
import torch.optim as optim
```

```
Epoch 8/10
```

```
import pandas as pd
import torch
```

```

import torch.nn as nn
import time
import os
from sklearn.metrics import f1_score

# Assuming create_model, validate, DEVICE, NUM_CLASSES, test_loader, criterion

def count_parameters(model):
    """Counts the total number of trainable parameters in a model."""
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

def evaluate_model_metrics(model_name):
    print(f"Evaluating {model_name}...")
    try:
        # Load the best model checkpoint
        checkpoint_path = f"results/checkpoints/{model_name}_best.pth"
        if not os.path.exists(checkpoint_path):
            print(f"Warning: Checkpoint not found for {model_name}. Skipping")
            return None

        model = create_model(model_name, NUM_CLASSES)
        model.load_state_dict(torch.load(checkpoint_path, map_location=DEVICE))
        model = model.to(DEVICE)
        model.eval()

        # Evaluate on test set
        test_loss, test_accuracy = validate(model, test_loader, criterion)

        # Calculate inference time
        start_time = time.time()
        with torch.no_grad():
            for images, _ in test_loader:
                images = images.to(DEVICE)
                _ = model(images)
        end_time = time.time()
        inference_time_per_image_ms = (end_time - start_time) / len(test_loader)

        # Count parameters
        total_parameters = count_parameters(model)

        # Calculate F1-score (macro average)
        all_preds = []
        all_labels = []
        with torch.no_grad():
            for images, labels in test_loader:
                images = images.to(DEVICE)
                outputs = model(images)
                _, predicted = torch.max(outputs, 1)
                all_preds.extend(predicted.cpu().numpy())
                all_labels.extend(labels.cpu().numpy())

```

```

f1 = f1_score(all_labels, all_preds, average='macro')

return {
    "Model": model_name,
    "Accuracy": test_accuracy,
    "F1-Score": f1,
    "Inference Time (ms)": inference_time_per_image_ms,
    "Total Parameters": total_parameters
}

except Exception as e:
    print(f"Error evaluating {model_name}: {e}")
    return None

# Collect results for all models
all_model_results = []
for model_name in MODEL_NAMES:
    result = evaluate_model_metrics(model_name)
    if result:
        all_model_results.append(result)

# Create DataFrame and save
if all_model_results:
    final_comparison = pd.DataFrame(all_model_results)
    final_comparison.to_csv("results/final_model_comparison.csv", index=False)
    print("\n'results/final_model_comparison.csv' created successfully!")
    print(final_comparison)
else:
    print("\nNo models were successfully evaluated to create 'results/final_model_comparison.csv'")
    final_comparison = pd.DataFrame() # Create an empty DataFrame to avoid

# Original discussion block (now conditional)
if not final_comparison.empty:
    best_accuracy_row = (
        final_comparison
        .sort_values(
            "Accuracy",
            ascending=False
        )
        .iloc[0]
    )

    best_f1_row = (
        final_comparison
        .sort_values(
            "F1-Score",
            ascending=False
        )
        .iloc[0]
    )

    fastest_row = (

```

```
        final_comparison
        .sort_values(
            "Inference Time (ms)"
        )
        .iloc[0]
    )

smallest_row = (
    final_comparison
    .sort_values(
        "Total Parameters"
    )
    .iloc[0]
```